

PAYTRACE

Payment Discrepancy Detection & Reconciliation System

FINTECH OPERATIONS • JAVA OOP ACADEMIC BLUEPRINT

Comprehensive Beginner Engineering Guide for Each Team Member

---

Team Division & Ownership Matrix

Member	Role	Assigned Git Branch	Core Deliverables
Thurika G	Domain Architect	`feature/java-data-models`	OOP class hierarchy, encapsulation, `BaseFinancialRecord`, `OrderRecord`, `PaymentTransaction`, `SettlementRecord`, `BigDecimal` precision
Sujith E	Core Systems Engineer	`feature/reconciliation-engine`	`ReconciliationEngine`, $O(N)$ HashMap matching, tolerance checks ($\pm\$0.05$), duplicate detection, variance calculations
Tharun J	Ingestion Specialist	`feature/csv-import`	`CsvImportService`, OpenCSV line tokenization, schema headers, non-blocking error handling, `CsvValidationResult`
Mithunasri J	QA & Workflow Engineer	`feature/investigation-testing`	`InvestigationCase` state machine (OPEN → UNDER_REVIEW → RESOLVED), mandatory audit notes, complete JUnit 5 test suite

> **Repository URL:** `https://github.com/sujith0718-coder/PayTrace.git`
> **Tech Stack:** Java 21 LTS • Spring Boot 3.4.1 • Maven • JUnit 5 • OpenCSV • React 18 + TypeScript
> **Design System:** Midnight Luxe (Deep Navy `#11182D`, Champagne Gold `#D6B66C`, Emerald `#26966B`)

MEMBER 1 GUIDE

Thurika G — Domain Architect

> **Primary Role:** Java OOP & Data Models
> **Git Branch:** `feature/java-data-models`
> **Target Package:** `com.paytrace.model` (in `backend/src/main/java/com/paytrace/model/`)

1. Your Objective

You build the **core domain entities** of PayTrace. Every other teammate (Sujith, Tharun, and Mithunasri) will import and use your classes to store, match, and validate financial records. You will demonstrate key OOP principles: **Abstraction, Inheritance, Encapsulation, and Polymorphism**.

2. Git & Workspace Setup

```
git clone https://github.com/sujith0718-coder/PayTrace.git
cd PayTrace
git checkout feature/java-data-models
```

Open the `backend/` folder in IntelliJ IDEA, Eclipse, or VS Code as a Maven project.

3. Step-by-Step Code Implementation

Step 1: `BaseFinancialRecord.java` (Abstract Parent Class)

```
package com.paytrace.model;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public abstract class BaseFinancialRecord {
    protected String recordId;
    protected LocalDateTime timestamp;
    protected BigDecimal amount;
    protected String currency;

    public BaseFinancialRecord(String recordId, LocalDateTime timestamp, BigDecimal amount, String c
urrency) {
        if (amount != null && amount.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Amount cannot be negative: " + amount);
        }
        this.recordId = recordId;
        this.timestamp = timestamp != null ? timestamp : LocalDateTime.now();
        this.amount = amount != null ? amount : BigDecimal.ZERO;
        this.currency = currency != null ? currency : "USD";
    }

    public String getRecordId() { return recordId; }
    public void setRecordId(String recordId) { this.recordId = recordId; }
    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp; }
    public BigDecimal getAmount() { return amount; }
```

```
    public void setAmount(BigDecimal amount) {
        if (amount != null && amount.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("Amount cannot be negative");
        }
        this.amount = amount;
    }
    public String getCurrency() { return currency; }
    public void setCurrency(String currency) { this.currency = currency; }

    public abstract String getSourceType();
}
```

Step 2: `OrderRecord.java` (OMS Order Child Class)

```
package com.paytrace.model;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class OrderRecord extends BaseFinancialRecord {
    private String customerId;
    private String orderStatus;

    public OrderRecord(String orderId, LocalDateTime orderDate, String customerId, BigDecimal amoun
t, String currency, String orderStatus) {
        super(orderId, orderDate, amount, currency);
        this.customerId = customerId;
        this.orderStatus = orderStatus;
    }

    public String getCustomerId() { return customerId; }
    public void setCustomerId(String customerId) { this.customerId = customerId; }
    public String getOrderStatus() { return orderStatus; }
    public void setOrderStatus(String orderStatus) { this.orderStatus = orderStatus; }

    @Override
    public String getSourceType() { return "ORDER_MANAGEMENT_SYSTEM"; }

    @Override
    public String toString() {
        return "OrderRecord[ID=" + recordId + ", Amount=" + amount + ", Customer=" + customerId +
"]";
    }
}
```

Step 3: `PaymentTransaction.java` (Gateway Payment Child Class)

```
package com.paytrace.model;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class PaymentTransaction extends BaseFinancialRecord {
    private String orderId;
    private BigDecimal feeAmount;
    private String gatewayName;
    private String status;

    public PaymentTransaction(String paymentId, String orderId, LocalDateTime paymentDate, BigDecima
l amount, BigDecimal feeAmount, String gatewayName, String status) {
        super(paymentId, paymentDate, amount, "USD");
```

```
        this.orderId = orderId;
        this.feeAmount = feeAmount != null ? feeAmount : BigDecimal.ZERO;
        this.gatewayName = gatewayName;
        this.status = status;
    }

    public String getOrderId() { return orderId; }
    public BigDecimal getFeeAmount() { return feeAmount; }
    public String getGatewayName() { return gatewayName; }
    public String getStatus() { return status; }

    @Override
    public String getSourceType() { return "PAYMENT_GATEWAY"; }
}
```

Step 4: `SettlementRecord.java` (Bank Settlement Child Class)

```
package com.paytrace.model;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class SettlementRecord extends BaseFinancialRecord {
    private String paymentId;
    private BigDecimal grossAmount;
    private BigDecimal netAmount;
    private BigDecimal bankFee;
    private String bankReference;

    public SettlementRecord(String settlementId, String paymentId, LocalDateTime settlementDate, Big
Decimal grossAmount, BigDecimal netAmount, BigDecimal bankFee, String bankReference) {
        super(settlementId, settlementDate, netAmount, "USD");
        this.paymentId = paymentId;
        this.grossAmount = grossAmount;
        this.netAmount = netAmount;
        this.bankFee = bankFee;
        this.bankReference = bankReference;
    }

    public String getPaymentId() { return paymentId; }
    public BigDecimal getGrossAmount() { return grossAmount; }
    public BigDecimal getNetAmount() { return netAmount; }
    public BigDecimal getBankFee() { return bankFee; }
    public String getBankReference() { return bankReference; }

    @Override
    public String getSourceType() { return "BANK_SETTLEMENT"; }
}
```

Step 5: `DiscrepancyType.java` & `Discrepancy.java`

```
package com.paytrace.model;

public enum DiscrepancyType {
    MATCHED, AMOUNT_MISMATCH, MISSING_PAYMENT, MISSING_SETTLEMENT, DUPLICATE_TRANSACTION, INVALID_FO
RMAT
}
```

```
package com.paytrace.model;
```

```
import java.math.BigDecimal;
import java.time.LocalDateTime;

public class Discrepancy {
    private String id;
    private DiscrepancyType type;
    private String orderId;
    private BigDecimal expectedAmount;
    private BigDecimal actualAmount;
    private BigDecimal variance;
    private String explanation;
    private LocalDateTime detectedAt;

    public Discrepancy(String id, DiscrepancyType type, String orderId, BigDecimal expectedAmount, B
igDecimal actualAmount, String explanation) {
        this.id = id;
        this.type = type;
        this.orderId = orderId;
        this.expectedAmount = expectedAmount != null ? expectedAmount : BigDecimal.ZERO;
        this.actualAmount = actualAmount != null ? actualAmount : BigDecimal.ZERO;
        this.variance = this.expectedAmount.subtract(this.actualAmount);
        this.explanation = explanation;
        this.detectedAt = LocalDateTime.now();
    }

    public String getId() { return id; }
    public DiscrepancyType getType() { return type; }
    public String getOrderId() { return orderId; }
    public BigDecimal getExpectedAmount() { return expectedAmount; }
    public BigDecimal getActualAmount() { return actualAmount; }
    public BigDecimal getVariance() { return variance; }
    public String getExplanation() { return explanation; }
    public LocalDateTime getDetectedAt() { return detectedAt; }
}
```

4. Viva / Interview Defense Preparation

Q: "Why did you use BigDecimal instead of double for transaction amounts?"

Answer: `double` uses binary floating-point representation (IEEE-754) which cannot represent decimal fractions like 0.1 exactly (e.g. 0.1 + 0.2 = 0.30000000000000004). In financial reconciliation, accumulating floating point errors leads to false discrepancies. `BigDecimal` provides exact arbitrary-precision arithmetic and exact rounding control.

Q: "What OOP principles are represented in your design?"

Answer: 1. **Abstraction:** `BaseFinancialRecord` defines common contracts and the abstract method `getSourceType()`.
2. **Inheritance:** `OrderRecord`, `PaymentTransaction`, and `SettlementRecord` inherit common fields using `super()`.
3. **Encapsulation:** Private/protected fields with validation checks inside getters/setters (e.g., rejecting negative values).

5. Git Commands to Save & Push

```
git add .
git commit -m "feat: implement domain models BaseFinancialRecord, OrderRecord, PaymentTransaction, S
```

```
ettlementRecord, Discrepancy"
git push origin feature/java-data-models
```

MEMBER 2 GUIDE

Sujith E — Core Systems Engineer

> **Primary Role:** Reconciliation Algorithm & Discrepancy Engine
> **Git Branch:** `feature/reconciliation-engine`
> **Target Package:** `com.paytrace.service` (in `backend/src/main/java/com/paytrace/service/`)

1. Your Objective

You build the **engine room of PayTrace**: the 3-Way Reconciliation Algorithm. You cross-compare Orders, Payments, and Bank Settlements, identify variances, handle a tolerance threshold ($\pm\$0.05$), and produce an immutable reconciliation result in **$O(N)$ linear time** using HashMaps.

2. Git & Workspace Setup

```
git clone https://github.com/sujith0718-coder/PayTrace.git
cd PayTrace
git checkout feature/reconciliation-engine
git pull origin main
```

3. Step-by-Step Code Implementation

Step 1: `ReconciliationResult.java` (Metrics Container)

```
package com.paytrace.service;

import com.paytrace.model.Discrepancy;
import java.math.BigDecimal;
import java.util.List;

public class ReconciliationResult {
    private int totalRecords;
    private int matchedRecords;
    private int discrepanciesCount;
    private BigDecimal totalOrderValue;
    private BigDecimal totalSettledValue;
    private List<Discrepancy> discrepancies;

    public ReconciliationResult(int totalRecords, int matchedRecords, List<Discrepancy> discrepancie
s, BigDecimal totalOrderValue, BigDecimal totalSettledValue) {
        this.totalRecords = totalRecords;
        this.matchedRecords = matchedRecords;
        this.discrepancies = discrepancies;
        this.discrepanciesCount = discrepancies.size();
        this.totalOrderValue = totalOrderValue;
        this.totalSettledValue = totalSettledValue;
    }

    public int getTotalRecords() { return totalRecords; }
    public int getMatchedRecords() { return matchedRecords; }
    public int getDiscrepanciesCount() { return discrepanciesCount; }
```

```
    public double getReconciliationPercentage() {
        if (totalRecords == 0) return 100.0;
        return ((double) matchedRecords / totalRecords) * 100.0;
    }
    public List<Discrepancy> getDiscrepancies() { return discrepancies; }
    public BigDecimal getTotalOrderValue() { return totalOrderValue; }
    public BigDecimal getTotalSettledValue() { return totalSettledValue; }
}
```

Step 2: `ReconciliationEngine.java` (The O(N) Algorithm)

```
package com.paytrace.service;

import com.paytrace.model.*;
import java.math.BigDecimal;
import java.util.*;

public class ReconciliationEngine {
    private final BigDecimal tolerance;

    public ReconciliationEngine() {
        this(new BigDecimal("0.05")); // Default 5 cents tolerance
    }

    public ReconciliationEngine(BigDecimal tolerance) {
        this.tolerance = tolerance;
    }

    public ReconciliationResult reconcile(
            List<OrderRecord> orders,
            List<PaymentTransaction> payments,
            List<SettlementRecord> settlements
    ) {
        List<Discrepancy> discrepancies = new ArrayList<>();
        int matchedCount = 0;
        BigDecimal totalOrderValue = BigDecimal.ZERO;
        BigDecimal totalSettledValue = BigDecimal.ZERO;

        // 1. Index payments by orderId into Map<String, List<PaymentTransaction>> to detect duplica
tes
        Map<String, List<PaymentTransaction>> paymentMap = new HashMap<>();
        for (PaymentTransaction p : payments) {
            paymentMap.computeIfAbsent(p.getOrderId(), k -> new ArrayList<>()).add(p);
        }

        // 2. Index settlements by paymentId into Map<String, SettlementRecord>
        Map<String, SettlementRecord> settlementMap = new HashMap<>();
        for (SettlementRecord s : settlements) {
            settlementMap.put(s.getPaymentId(), s);
            totalSettledValue = totalSettledValue.add(s.getNetAmount());
        }

        // 3. 3-Way Match Loop over Orders
        int discIdCounter = 100;
        for (OrderRecord order : orders) {
            totalOrderValue = totalOrderValue.add(order.getAmount());
            String orderId = order.getRecordId();
            List<PaymentTransaction> matchingPayments = paymentMap.get(orderId);

            // Case A: Missing Payment
            if (matchingPayments == null || matchingPayments.isEmpty()) {
                discrepancies.add(new Discrepancy(
```

```
                    "DISC-" + (++discIdCounter), DiscrepancyType.MISSING_PAYMENT,
                    orderId, order.getAmount(), BigDecimal.ZERO,
                    "Order exists in OMS, but no payment gateway transaction was captured."
                ));
                continue;
            }

            // Case B: Duplicate Transactions
            if (matchingPayments.size() > 1) {
                BigDecimal totalPaid = matchingPayments.stream()
                        .map(PaymentTransaction::getAmount)
                        .reduce(BigDecimal.ZERO, BigDecimal::add);
                discrepancies.add(new Discrepancy(
                    "DISC-" + (++discIdCounter), DiscrepancyType.DUPLICATE_TRANSACTION,
                    orderId, order.getAmount(), totalPaid,
                    "Multiple payments (" + matchingPayments.size() + ") detected for single order
#" + orderId
                ));
                continue;
            }

            PaymentTransaction payment = matchingPayments.get(0);

            // Case C: Amount Mismatch
            BigDecimal amountDiff = order.getAmount().subtract(payment.getAmount()).abs();
            if (amountDiff.compareTo(tolerance) > 0) {
                discrepancies.add(new Discrepancy(
                    "DISC-" + (++discIdCounter), DiscrepancyType.AMOUNT_MISMATCH,
                    orderId, order.getAmount(), payment.getAmount(),
                    "Order amount ($" + order.getAmount() + ") does not match gateway amount ($" + p
ayment.getAmount() + ")."
                ));
                continue;
            }

            // Case D: Missing Settlement
            SettlementRecord settlement = settlementMap.get(payment.getRecordId());
            if (settlement == null) {
                discrepancies.add(new Discrepancy(
                    "DISC-" + (++discIdCounter), DiscrepancyType.MISSING_SETTLEMENT,
                    orderId, payment.getAmount(), BigDecimal.ZERO,
                    "Payment captured on gateway, but bank settlement record has not settled."
                ));
                continue;
            }

            matchedCount++;
        }

        return new ReconciliationResult(orders.size(), matchedCount, discrepancies, totalOrderValue,
totalSettledValue);
    }
}
```

4. Viva / Interview Defense Preparation

> **Q: "How does your matching algorithm avoid an O(N^3) performance bottleneck?"**
>
> **Answer:** "If we used nested loops (checking every order against every payment and settlement), the time complexity would be $O(N \times M \times K)$, which for 2,000 records requires $8 \text{ billion comparisons}$ and causes the JVM to hang. By pre-indexing payments and settlements into in-memory HashMaps keyed by

`orderId` and `paymentId`, lookups happen in constant $O(1)$ time, reducing overall algorithm complexity to linear $O(N)$."

Q: "Why did you use Map<String, List<PaymentTransaction>> instead of Map<String, PaymentTransaction>?"

Answer: "If a customer accidentally clicks 'Submit Payment' twice and gets double-charged, mapping to a single value would overwrite the first payment and hide the bug. Mapping to a `List` allows us to detect when the size is greater than 1 and flag a `DUPLICATE_TRANSACTION`."

5. Git Commands to Save & Push

```
git add .
git commit -m "feat: implement O(N) HashMap 3-way reconciliation engine with tolerance checks"
git push origin feature/reconciliation-engine
```

MEMBER 3 GUIDE

Tharun J — Ingestion Specialist

> **Primary Role:** CSV Ingestion, Parsing & Non-blocking Validation
> **Git Branch:** `feature/csv-import`
> **Target Package:** `com.paytrace.ingestion` (in `backend/src/main/java/com/paytrace/ingestion/`)

1. Your Objective

You build the **data gateway** of PayTrace. You ingest three CSV files (Orders, Payments, Settlements). Your parser must tokenize rows, cast strings to `BigDecimal` and dates, validate required headers, and **never crash** if row 45 contains corrupted data—capturing warnings and continuing parsing smoothly.

2. Git & Workspace Setup

```
git clone https://github.com/sujith0718-coder/PayTrace.git
cd PayTrace
git checkout feature/csv-import
git pull origin main
```

3. Step-by-Step Code Implementation

Step 1: `CsvValidationResult.java` (Generic Parser Envelope)

```
package com.paytrace.ingestion;

import java.util.ArrayList;
import java.util.List;

public class CsvValidationResult<T> {
    private List<T> validRecords = new ArrayList<>();
    private List<String> warnings = new ArrayList<>();
    private int invalidRowsCount = 0;

    public void addRecord(T record) { validRecords.add(record); }
    public void addWarning(String warning) {
        warnings.add(warning);
        invalidRowsCount++;
    }

    public List<T> getValidRecords() { return validRecords; }
    public List<String> getWarnings() { return warnings; }
    public int getInvalidRowsCount() { return invalidRowsCount; }
    public int getTotalRowsParsed() { return validRecords.size() + invalidRowsCount; }
}
```

Step 2: `CsvImportService.java` (OpenCSV Parser Engine)

```
package com.paytrace.ingestion;
```

```
import com.opencsv.CSVReader;
import com.paytrace.model.*;
import java.io.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

public class CsvImportService {

    public CsvValidationResult<OrderRecord> parseOrders(InputStream inputStream) {
        CsvValidationResult<OrderRecord> result = new CsvValidationResult<>();
        try (CSVReader reader = new CSVReader(new InputStreamReader(inputStream))) {
            String[] line;
            int lineNumber = 0;
            while ((line = reader.readNext()) != null) {
                lineNumber++;
                if (lineNumber == 1) continue; // Skip header

                if (line.length < 4) {
                    result.addWarning("Line " + lineNumber + ": Insufficient columns");
                    continue;
                }
                try {
                    String orderId = line[0].trim();
                    String customerId = line[2].trim();
                    BigDecimal amount = new BigDecimal(line[3].trim());
                    String currency = line.length > 4 ? line[4].trim() : "USD";
                    String status = line.length > 5 ? line[5].trim() : "COMPLETED";

                    if (orderId.isEmpty()) {
                        result.addWarning("Line " + lineNumber + ": Missing order_id");
                        continue;
                    }
                    result.addRecord(new OrderRecord(orderId, LocalDateTime.now(), customerId, amoun
t, currency, status));
                } catch (NumberFormatException e) {
                    result.addWarning("Line " + lineNumber + ": Invalid numeric amount '" + line[3]
+ "'");
                } catch (Exception e) {
                    result.addWarning("Line " + lineNumber + ": Corrupted row - " + e.getMessage());
                }
            }
        } catch (Exception e) {
            result.addWarning("Fatal CSV read error: " + e.getMessage());
        }
        return result;
    }

    public CsvValidationResult<PaymentTransaction> parsePayments(InputStream inputStream) {
        CsvValidationResult<PaymentTransaction> result = new CsvValidationResult<>();
        try (CSVReader reader = new CSVReader(new InputStreamReader(inputStream))) {
            String[] line;
            int lineNumber = 0;
            while ((line = reader.readNext()) != null) {
                lineNumber++;
                if (lineNumber == 1) continue;
                try {
                    String paymentId = line[0].trim();
                    String orderId = line[1].trim();
                    BigDecimal amount = new BigDecimal(line[3].trim());
                    BigDecimal fee = line.length > 4 ? new BigDecimal(line[4].trim()) : BigDecimal.Z
ERO;
                    String status = line.length > 5 ? line[5].trim() : "CAPTURED";
                    String gateway = line.length > 6 ? line[6].trim() : "Stripe";
```

```
                    result.addRecord(new PaymentTransaction(paymentId, orderId, LocalDateTime.now(),
amount, fee, gateway, status));
                } catch (Exception e) {
                    result.addWarning("Line " + lineNumber + ": Corrupted payment row");
                }
            }
        } catch (Exception e) {
            result.addWarning("Fatal CSV read error: " + e.getMessage());
        }
        return result;
    }

    public CsvValidationResult<SettlementRecord> parseSettlements(InputStream inputStream) {
        CsvValidationResult<SettlementRecord> result = new CsvValidationResult<>();
        try (CSVReader reader = new CSVReader(new InputStreamReader(inputStream))) {
            String[] line;
            int lineNumber = 0;
            while ((line = reader.readNext()) != null) {
                lineNumber++;
                if (lineNumber == 1) continue;
                try {
                    String settlementId = line[0].trim();
                    String paymentId = line[1].trim();
                    BigDecimal gross = new BigDecimal(line[3].trim());
                    BigDecimal net = new BigDecimal(line[4].trim());
                    BigDecimal fee = line.length > 5 ? new BigDecimal(line[5].trim()) : BigDecimal.Z
ERO;
                    String ref = line.length > 6 ? line[6].trim() : "BANK-REF";

                    result.addRecord(new SettlementRecord(settlementId, paymentId, LocalDateTime.now
(), gross, net, fee, ref));
                } catch (Exception e) {
                    result.addWarning("Line " + lineNumber + ": Corrupted settlement row");
                }
            }
        } catch (Exception e) {
            result.addWarning("Fatal CSV read error: " + e.getMessage());
        }
        return result;
    }
}
```

4. Viva / Interview Defense Preparation

Q: "How does your ingestion pipeline prevent a single corrupt row from failing an entire 10,000-row file?"

Answer: "I implemented fine-grained, row-level try-catch exception handling. Instead of throwing an unchecked exception that aborts the transaction, any `NumberFormatException` or missing field is isolated into a `ValidationWarning` log with the exact line number. The loop continues, preserving all valid records."

Q: "Why did you use InputStream instead of java.io.File?"

Answer: "In modern Spring Boot web applications, client uploads arrive via HTTP as `MultipartFile`, which streams input directly in memory as an `InputStream`. Using `InputStream` prevents costly disk I/O and works seamlessly for unit testing."

5. Git Commands to Save & Push

```
git add .
git commit -m "feat: implement CsvImportService with OpenCSV and non-blocking line validation"
git push origin feature/csv-import
```

MEMBER 4 GUIDE

Mithunasri J — QA & Workflow Engineer

> **Primary Role:** Investigation State Machine & JUnit 5 Testing Suite
> **Git Branch:** `feature/investigation-testing`
> **Target Packages:** `com.paytrace.workflow` & `com.paytrace` (tests)

1. Your Objective

You build the **Resolution Workflow and Quality Assurance** of PayTrace: 1. Build the **Investigation Case State Machine**: Discrepancy → `OPEN` → `UNDER_REVIEW` → `RESOLVED` (with mandatory resolution category and audit note). 2. Author the **JUnit 5 test suite** that validates Thurika's models, Sujith's algorithm, and Tharun's CSV reader.

2. Git & Workspace Setup

```
git clone https://github.com/sujith0718-coder/PayTrace.git
cd PayTrace
git checkout feature/investigation-testing
git pull origin main
```

3. Step-by-Step Code Implementation

Step 1: `InvestigationStatus.java` & `InvestigationCase.java`

```
package com.paytrace.workflow;

public enum InvestigationStatus {
    OPEN, UNDER_REVIEW, RESOLVED, REOPENED
}
```

```
package com.paytrace.workflow;

import com.paytrace.model.Discrepancy;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class InvestigationCase {
    private String caseId;
    private Discrepancy discrepancy;
    private InvestigationStatus status;
    private String assignedAnalyst;
    private String resolutionCategory;
    private String resolutionNote;
    private LocalDateTime createdAt;
    private LocalDateTime resolvedAt;
    private List<String> notes = new ArrayList<>();

    public InvestigationCase(String caseId, Discrepancy discrepancy, String assignedAnalyst) {
        this.caseId = caseId;
```

```
        this.discrepancy = discrepancy;
        this.assignedAnalyst = assignedAnalyst;
        this.status = InvestigationStatus.OPEN;
        this.createdAt = LocalDateTime.now();
    }

    public void moveToUnderReview() {
        if (this.status == InvestigationStatus.RESOLVED) {
            throw new IllegalStateException("Cannot move a RESOLVED case to UNDER_REVIEW directly. R
eopen first.");
        }
        this.status = InvestigationStatus.UNDER_REVIEW;
    }

    public void resolve(String category, String note) {
        if (note == null || note.trim().isEmpty()) {
            throw new IllegalArgumentException("Resolution note is mandatory.");
        }
        this.status = InvestigationStatus.RESOLVED;
        this.resolutionCategory = category;
        this.resolutionNote = note;
        this.resolvedAt = LocalDateTime.now();
        this.notes.add("RESOLVED [" + category + "]: " + note);
    }

    public void reopen(String reason) {
        this.status = InvestigationStatus.REOPENED;
        this.notes.add("REOPENED: " + reason);
    }

    public void addNote(String author, String text) {
        this.notes.add(author + " (" + LocalDateTime.now() + "): " + text);
    }

    public String getCaseId() { return caseId; }
    public Discrepancy getDiscrepancy() { return discrepancy; }
    public InvestigationStatus getStatus() { return status; }
    public String getAssignedAnalyst() { return assignedAnalyst; }
    public List<String> getNotes() { return notes; }
    public String getResolutionCategory() { return resolutionCategory; }
}
```

Step 2: `ReconciliationEngineTest.java` (JUnit 5 Test Suite)

Put this file in `backend/src/test/java/com/paytrace/ReconciliationEngineTest.java`:

```
package com.paytrace;

import com.paytrace.model.*;
import com.paytrace.service.*;
import com.paytrace.workflow.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

public class ReconciliationEngineTest {
```

```
    private ReconciliationEngine engine;

    @BeforeEach
    void setUp() {
        engine = new ReconciliationEngine(new BigDecimal("0.05"));
    }

    @Test
    @DisplayName("Test 1: Perfectly balanced 3-way match")
    void testPerfectMatch() {
        OrderRecord order = new OrderRecord("ORD-1", LocalDateTime.now(), "CUST-1", new BigDecimal
("100.00"), "USD", "COMPLETED");
        PaymentTransaction payment = new PaymentTransaction("PAY-1", "ORD-1", LocalDateTime.now(), n
ew BigDecimal("100.00"), new BigDecimal("2.00"), "Stripe", "CAPTURED");
        SettlementRecord settlement = new SettlementRecord("SET-1", "PAY-1", LocalDateTime.now(), ne
w BigDecimal("100.00"), new BigDecimal("98.00"), new BigDecimal("2.00"), "BANK-1");

        ReconciliationResult result = engine.reconcile(List.of(order), List.of(payment), List.of(set
tlement));

        assertEquals(1, result.getMatchedRecords());
        assertEquals(0, result.getDiscrepanciesCount());
        assertEquals(100.0, result.getReconciliationPercentage());
    }

    @Test
    @DisplayName("Test 2: Amount mismatch outside tolerance ($250 vs $230)")
    void testAmountMismatch() {
        OrderRecord order = new OrderRecord("ORD-2", LocalDateTime.now(), "CUST-2", new BigDecimal
("250.00"), "USD", "COMPLETED");
        PaymentTransaction payment = new PaymentTransaction("PAY-2", "ORD-2", LocalDateTime.now(), n
ew BigDecimal("230.00"), new BigDecimal("4.00"), "Stripe", "CAPTURED");
        SettlementRecord settlement = new SettlementRecord("SET-2", "PAY-2", LocalDateTime.now(), ne
w BigDecimal("230.00"), new BigDecimal("226.00"), new BigDecimal("4.00"), "BANK-2");

        ReconciliationResult result = engine.reconcile(List.of(order), List.of(payment), List.of(set
tlement));

        assertEquals(0, result.getMatchedRecords());
        assertEquals(1, result.getDiscrepanciesCount());
        assertEquals(DiscrepancyType.AMOUNT_MISMATCH, result.getDiscrepancies().get(0).getType());
        assertEquals(new BigDecimal("20.00"), result.getDiscrepancies().get(0).getVariance());
    }

    @Test
    @DisplayName("Test 3: Missing payment gateway record")
    void testMissingPayment() {
        OrderRecord order = new OrderRecord("ORD-3", LocalDateTime.now(), "CUST-3", new BigDecimal
("150.00"), "USD", "COMPLETED");
        ReconciliationResult result = engine.reconcile(List.of(order), List.of(), List.of());

        assertEquals(1, result.getDiscrepanciesCount());
        assertEquals(DiscrepancyType.MISSING_PAYMENT, result.getDiscrepancies().get(0).getType());
    }

    @Test
    @DisplayName("Test 4: Investigation state machine transition and mandatory note")
    void testInvestigationWorkflow() {
        Discrepancy disc = new Discrepancy("DISC-1", DiscrepancyType.AMOUNT_MISMATCH, "ORD-1", new B
igDecimal("100"), new BigDecimal("90"), "Variance");
        InvestigationCase inv = new InvestigationCase("INV-101", disc, "Mithunasri J");

        assertEquals(InvestigationStatus.OPEN, inv.getStatus());
        inv.moveToUnderReview();
```

```
        assertEquals(InvestigationStatus.UNDER_REVIEW, inv.getStatus());

        assertThrows(IllegalArgumentException.class, () -> inv.resolve("TIMING_DIFFERENCE", "   "));

        inv.resolve("TIMING_DIFFERENCE", "Verified customer promotional credit.");
        assertEquals(InvestigationStatus.RESOLVED, inv.getStatus());
        assertEquals("TIMING_DIFFERENCE", inv.getResolutionCategory());
    }
}
```

4. Viva / Interview Defense Preparation

> **Q: "Why did you write automated unit tests instead of only testing through the browser?"**
>
> **Answer:** "Automated JUnit 5 tests provide deterministic regression testing. When another teammate modifies a class, running `mvn test` validates in seconds whether edge cases—like amount tolerances, missing payments, or state transitions—still function properly without having to manually upload files."

> **Q: "Why is a note required when resolving an investigation case?"**
>
> **Answer:** "In financial software auditability, closing an unexplained revenue variance without documentation is a compliance risk. Our method throws an `IllegalArgumentException` if the explanation string is empty, enforcing accountability."

5. Git Commands to Save & Push

```
git add .
git commit -m "feat: implement investigation case workflow and JUnit 5 regression test suite"
git push origin feature/investigation-testing
```